PROGRAMACIÓN AVANZADA

Sesión 15

Paso por Referencia

→ NOTAS DE CLASE

Índice

1 La trampa invisible **2**

2 Paso por valor: el comportamiento por defecto **2**

3 Paso por referencia **3**

4 Paso por apuntador: el mecanismo heredado de C **3**

5 ¿Cuándo usar cada mecanismo? **4**
5.1 Referencia constante: leer sin copiar 4

6 Funciones con múltiples resultados **5**

7 Arreglos: siempre por referencia implícita **6**
7.1 Arreglos bidimensionales 6

8 Aplicación: Bubble Sort **7**
8.1 La función `swap` 7
8.2 El algoritmo completo 7

9 Ejercicios **8**

10 Resumen **9**

La trampa invisible

Escribe esta función y dime qué esperas que imprima:

```
void sumaUno(int n) {
    n = n + 1;
}

int a = 10;
sumaUno(a);
cout << a;    // 10 u 11?
```

Si respondiste 11, estás en buena compañía — y en el error. El programa imprime **10**. La variable `a` no cambió. La función modificó una copia temporal y esa copia desapareció al terminar la llamada.

Esta sesión responde exactamente esa pregunta: ¿cómo le digo a una función que sí quiero que modifique el original?

> En **FORTRAN** (1957), el primer lenguaje de alto nivel, *todos* los argumentos se pasaban por referencia — incluso los literales numéricos. Esto llevó a un bug legendario: al escribir `CALL FOO(2)` y que `FOO` modificara su parámetro, el programa podía alterar el valor almacenado para el literal `2`. Después de la llamada, cualquier aparición de `2` en el programa devolvía `3`. Este tipo de aliasing involuntario fue una de las razones por las que C eligió el paso por valor como comportamiento predeterminado.
>
> *Referencia:* Backus, J. (1978). "The History of FORTRAN I, II, and III". *ACM SIGPLAN Notices*, 13(8), 165–180. https://doi.org/10.1145/960118.808380

Paso por valor: el comportamiento por defecto

Cuando llamas a una función, C++ crea una nueva variable local en el *stack* de esa función y **copia** el valor del argumento en ella. La función trabaja con esa copia; el original permanece intacto.

En `main()`	**En** `sumaUno()`

`+---------+`
`|  10  |  <- int a`
`+---------+`
`0xAA00`
	`+---------+`
	`|  10  |  <- int n  (copia)`
	`+---------+`
	`0xBB00`

`a` y `n` son celdas de memoria distintas. Cualquier cambio en `n` no afecta a `a`.

✔ Recuerda

El paso por valor es seguro y predecible: la función nunca modifica los datos del llamador por accidente. Es la opción correcta para datos pequeños (`int` , `double` , `char`) que la función solo necesita *leer*.

Paso por referencia

Para que una función **modifique** el original, declara el parámetro con `&` . El símbolo indica que la función recibe un *alias* — otro nombre para la misma celda de memoria — en lugar de una copia.

```
void sumaUno(int& n) {    // n es alias del argumento
    n = n + 1;
}

int a = 10;
cout << "antes:  " << a << endl;    // 10
sumaUno(a);
cout << "despues: " << a << endl;   // 11
```

Listing 1: Paso por referencia: el & en el parámetro

Ahora `n` y `a` son el mismo espacio de memoria:

Paso por valor	Paso por referencia
`main()` `+------+` `| 10 |` `+------+` `0xAA00`	`sumaUno()` `+------+` `| 10 | (copia)` `+------+` `0xBB00`
	`main()` `+------+` `| 10 | <-- a == n (mismo espacio)` `+------+` `0xAA00`

El `&` en una **declaración de parámetro** (`int& n`) indica paso por referencia. El `&` en una **expresión** (`&a`) es el operador dirección-de, que devuelve la dirección de `a` . Son dos usos del mismo símbolo con significados completamente distintos. En la *llamada* a la función no se escribe ningún `&` .

Paso por apuntador: el mecanismo heredado de C

Antes de que C++ introdujera las referencias, la única forma de lograr el mismo efecto era pasando la **dirección** de la variable y trabajando con un apuntador dentro de la función. C todavía usa este mecanismo hoy.

```
void sumaUno(int* p) {    // p recibe una direccion
```

```
    (*p) = (*p) + 1;        // desreferenciamos para modificar el original
}

int a = 10;
sumaUno(&a);                // pasamos la direccion de a
cout << a;                  // 11
```

Listing 2: Paso por apuntador

El resultado es idéntico al paso por referencia, pero la sintaxis es más ruidosa: hay que escribir `&a` en la llamada y `*p` dentro de la función.

`scanf` `&x`

Si alguna vez te preguntaste por qué `scanf("%d", &x)` exige el `&` y `cin » x` no, aquí está la respuesta: `scanf` es una función de C (que no tiene referencias), así que necesita la dirección para poder modificar `x`. `cin »` es un operador de C++ implementado con referencias, por lo que la dirección se pasa de forma implícita y transparente. El mecanismo es el mismo; solo cambia si el compilador lo hace visible o no.

¿Cuándo usar cada mecanismo?

Mecanismo	Sintaxis	Cuándo usarlo
Por valor	`int n`	Dato pequeño que la función solo lee
Por referencia	`int& n`	La función necesita modificar el original
Por ref. constante	`const int& n`	Dato grande que la función solo lee (evita copias)
Por apuntador	`int* p`	Puede ser `nullptr`, o al trabajar con arreglos y memoria dinámica

• Referencia constante: leer sin copiar

Un `std::string` o un `std::vector` pueden ocupar kilobytes o más. Pasarlos por valor crea una copia completa en cada llamada. La solución es `const&`: le prometemos al compilador que no vamos a modificar el dato, y a cambio solo pasamos la dirección (8 bytes), no una copia.

```
// MAL: crea una copia del string completo en cada llamada
void imprimirV(string texto) {
    cout << texto << endl;
}

// BIEN: solo pasa la direccion, sin copiar nada
void imprimir(const string& texto) {
    // texto = "otro";  // error: no se puede modificar (es const)
```

```
    cout << texto << endl;
}

string saludo = "Hola, mundo";
imprimir(saludo);   // eficiente: 8 bytes pasados, no toda la cadena
```

Listing 3: const& para evitar copias costosas

Regla práctica: si el parámetro es de un tipo que ocupa más de 8 bytes y la función no necesita modificarlo, usa `const&`. Para tipos primitivos (`int`, `double`, `char`) el valor copiado es tan pequeño como la dirección, así que el paso por valor es igual de eficiente y más claro.

Funciones con múltiples resultados

Una función solo puede tener un `return`. El paso por referencia permite "devolver" varios resultados usando **parámetros de salida**.

```
void minMax(const int* arr, int n, int& minVal, int& maxVal) {
    minVal = maxVal = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < minVal) minVal = arr[i];
        if (arr[i] > maxVal) maxVal = arr[i];
    }
}

int datos[6] = {3, 7, 1, 9, 4, 6};
int minimo, maximo;

minMax(datos, 6, minimo, maximo);

cout << "Minimo: " << minimo << endl;  // 1
cout << "Maximo: " << maximo << endl;  // 9
```

Listing 4: Mínimo y máximo de un arreglo en una sola pasada

`minVal` y `maxVal` son aliases de `minimo` y `maximo`. Al escribir en ellas dentro de la función, estamos escribiendo directamente en las variables del llamador.

Este patrón de parámetros de salida es omnipresente:

- La función `std::minmax_element` de la STL regresa un par de iteradores usando exactamente este concepto.
- En motores de videojuegos (Unreal Engine, Unity via C#), las funciones que calculan física o geometría frecuentemente devuelven múltiples vectores y

normales como parámetros `out`.

- En sistemas embebidos, donde las excepciones están deshabilitadas, los parámetros de salida son la forma estándar de reportar éxito/error junto con el resultado.

Arreglos: siempre por referencia implícita

Los arreglos son un caso especial: **siempre** se pasan por referencia, aunque no escribas ningún `&`. Al pasar un arreglo a una función, su nombre *decae* automáticamente a un apuntador al primer elemento, y la función trabaja directamente sobre el arreglo original.

```
void duplicar(int* arr, int n) {
    for (int i = 0; i < n; i++)
        arr[i] *= 2;    // modifica el arreglo original
}

int nums[5] = {1, 2, 3, 4, 5};
duplicar(nums, 5);

for (int i = 0; i < 5; i++)
    cout << nums[i] << " ";    // 2 4 6 8 10
```

Listing 5: Arreglos: la función modifica el original

✔ Recuerda

Cuando pasas un arreglo a una función siempre hay que pasar también el **tamaño** como parámetro adicional. Al decaer a apuntador, la información del tamaño se pierde: dentro de la función, `sizeof(arr)` daría el tamaño del apuntador (8 bytes), no del arreglo. Esto es exactamente lo que estudiamos con el *array decay* en la sesión anterior.

• Arreglos bidimensionales

Para pasar una matriz, la función necesita conocer **todas las dimensiones excepto la primera** (que puede omitirse):

```
void incrementar(int mat[][3], int filas) {
    for (int i = 0; i < filas; i++)
        for (int j = 0; j < 3; j++)
            mat[i][j]++;
}

int m[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
incrementar(m, 3);
// m ahora contiene {{2,3,4},{5,6,7},{8,9,10}}
```

Listing 6: Arreglo 2D como parámetro

La razón por la que se necesita la segunda dimensión es aritmética de apuntadores: para calcular `mat[i][j]`, el compilador necesita saber cuántos `int` ocupa cada fila.

Aplicación: Bubble Sort

Bubble Sort es el ejemplo perfecto para conectar los tres conceptos de esta sesión: el `swap` requiere paso por referencia, el `bubblesort` recibe el arreglo como apuntador, y el `comparar` recibe por valor (solo lee).

• La función `swap`

Intercambiar dos variables es imposible con paso por valor. Necesitamos referencias:

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int x = 5, y = 9;
swap(x, y);
cout << x << " " << y;  // 9 5
```

Listing 7: swap: el ejemplo canónico de paso por referencia

> La biblioteca estándar ya incluye `std::swap` (en `<algorithm>`), implementado exactamente así. Internamente, todos los algoritmos de ordenamiento de la STL (`std::sort`, `std::stable_sort`) la usan.

• El algoritmo completo

La idea de Bubble Sort: compara pares adyacentes e intercámbialos si están en el orden equivocado. Repite hasta que ningún par necesite intercambio.

Pasada	Arreglo	Acción
Inicial	`[5, 3, 8, 1, 4]`	—
1	`[3, 5, 1, 4, 8]`	el 8 llegó al final
2	`[3, 1, 4, 5, 8]`	el 5 llegó a su lugar
3	`[1, 3, 4, 5, 8]`	el 3 llegó a su lugar
4	`[1, 3, 4, 5, 8]`	sin cambios: terminó

El nombre viene de que los elementos grandes “burbujean” hacia el final en cada pasada.

```
bool comparar(int a, int b) {
    return a > b;   // true si hay que intercambiar (a
        deberia ir despues)
```

```
}

void swap(int& a, int& b) {
    int temp = a;  a = b;  b = temp;
}

void bubblesort(int* arr, int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (comparar(arr[j], arr[j+1]))
                swap(arr[j], arr[j+1]);
}
```

Listing 8: Bubble Sort completo

std::sort

La función `std::sort` de la STL acepta un **comparador** como tercer argumento — exactamente igual que nuestro `comparar`. Esto te permite ordenar en cualquier dirección o por cualquier criterio:

```
#include <algorithm>
int arr[5] = {3, 1, 4, 1, 5};

// Orden ascendente (defecto)
sort(arr, arr+5);

// Orden descendente: pasa una funcion comparadora
sort(arr, arr+5, [](int a, int b){ return a > b; });
```

La *lambda* `[](int a, int b){ return a > b; }` es básicamente nuestra función `comparar` escrita en línea. C++11 la introdujo precisamente para evitar tener que declarar funciones auxiliares por separado.

Ejercicios

• Ejercicio 1 — Elevar al cubo

Escribe una función `cubo(int& n)` que eleve `n` a la tercera potencia, modificando la variable original. Pruébala con los valores $\{2, 3, 4, 5\}$ e imprime el arreglo antes y después de aplicarla a cada elemento.

• Ejercicio 2 — Tres resultados de una sola pasada

Escribe una función:

```
void estadisticas(const int* arr, int n,
                  int& minVal, int& maxVal, int& suma);
```

que calcule el mínimo, el máximo y la suma del arreglo en una sola pasada. Pruébala con `int datos[7] = {12, 5, 8, 21, 3, 17, 9}`.

- ## Ejercicio 3 — Normalizar un arreglo

Escribe una función `normalizar(double* arr, int n)` que divida cada elemento entre el valor máximo del arreglo, de modo que todos los valores queden en el rango $[0, 1]$. Pruébala con `double v[5] = {20.0, 50.0, 10.0, 80.0, 35.0}`.
Pista: necesitarás encontrar el máximo primero y luego recorrer el arreglo de nuevo. ¿Podrías reutilizar tu función del ejercicio 2?

- ## Ejercicio 4 — Bubble Sort descendente

Modifica el `comparar` del Bubble Sort para que el algoritmo ordene en orden **descendente**. Sin cambiar `swap` ni `bubblesort`, ¿qué línea es la única que hay que modificar? Explica por escrito por qué basta con ese cambio.

Resumen

✔ Recuerda

- Por **valor** (`int n`): la función recibe una copia; el original no cambia.
- Por **referencia** (`int& n`): la función recibe un alias; sí modifica el original.
- Por **referencia constante** (`const int& n`): alias de solo lectura; útil para objetos grandes.
- Por **apuntador** (`int* p`): la función recibe una dirección; puede ser `nullptr`.
- Los arreglos **siempre** decaen a apuntador al pasarse; hay que pasar el tamaño por separado.
- Con referencias se pueden devolver **múltiples resultados** de una función sin usar `return`.
- `swap` con referencias es el patrón base de todos los algoritmos de ordenamiento.
- `std::sort` acepta un comparador para ordenar por cualquier criterio.